# Spatio-Temporal Predictive Navigation & Dynamic Obstacle Avoidance for Autonomous UAVs using ConvLSTM

*Advanced Deep Learning & Autonomous Control Coursework Problem Statement & Dataset Guide*

| | |
|---|---|
| **Target Audience:** | M.Tech (Robotics, Autonomous Systems, AI, Data Science) |
| **Core Prerequisites:** | PyTorch / TensorFlow, ROS 2 / PX4 Autopilot, Convolutional Neural Networks, Recurrent Architectures |
| **Project Duration:** | 8 Weeks (Intermediate Capstone / Thesis Module) |
| **Primary Modality:** | Spatio-Temporal Forecasting + Software-In-The-Loop (SITL) Closed-Loop Flight Simulation |

> **Project Assignment Summary**
> Students must build an end-to-end autonomous UAV flight navigation pipeline that forecasts dynamic obstacle occupancy using a Convolutional Long Short-Term Memory (ConvLSTM) architecture and generates real-time collision-free velocity commands towards target GPS locations.

## 1. Executive Summary & Problem Scope

Autonomous Unmanned Aerial Vehicles (UAVs) operating in low-altitude urban environments or dense unstructured environments (e.g., forests, construction sites) face severe navigation challenges from both static structures (buildings, powerlines, trees) and dynamic moving entities (other aircraft, ground vehicles, birds). Standard point-cloud or reactive potential-field navigation methods suffer from a lack of temporal awareness, resulting in jerky control actions, local minima entrapment, or catastrophic collisions when paths are blocked by dynamic obstacles.

Standard LSTM networks process vector-based time series but discard vital spatial topology when flattening image/grid representations. To overcome this, this project leverages a Convolutional LSTM (ConvLSTM) architecture. By replacing matrix multiplication operations with convolutional structures inside the LSTM cell state transitions, the model simultaneously models spatial visual structure (where obstacles are located) and temporal motion dynamics (where moving obstacles will be in future time steps).

## 2. Problem Formulation & Mathematical Foundation

Let the UAV kinematic state at discrete time step t be defined as:

$$s_t = [\ x_t,\ \ y_t,\ \ z_t,\ \ v_{x,t},\ \ v_{y,t},\ \ v_{z,t},\ \ \psi_t\ ]^T$$

Where $(x_t, y_t, z_t)$ represents 3D world position, $(v_x, v_y, v_z)$ denotes linear velocity components, and $\psi_t$ represents yaw angle. Given a user-specified global target location $g = [x_g, y_g, z_g]^T$ and a sequential lookback window of k egocentric spatial Bird's-Eye-View (BEV) occupancy grid maps:

$$O_{t-k:t} = \{ O_{t-k}, O_{t-k+1}, ..., O_t \}, \quad \text{where } O_t \in \mathbb{R}^{H \times W \times C}$$

The student must design a dual-objective system:

- **Spatio-Temporal Perception Objective:** Train a ConvLSTM model $f_\theta$ to predict future spatial occupancy maps across a p-step future horizon: $\hat{O}_{t+1:t+p} = f_\theta(O_{t-k:t})$.
- **Control Objective:** Output kinodynamically feasible velocity control commands $u_t = [v_{x,cmd}, v_{y,cmd}, v_{z,cmd}, \dot{\psi}_{cmd}]$ that minimize Euclidean distance to target goal g while enforcing a minimum clearance buffer $d_{min} > 1.5$ meters from all predicted obstacle occupancy cells.

## 3. System Architecture & Core Modules

The student's implementation must adhere to a modular architecture combining vision-based grid encoding, temporal forecasting, waypoint generation, and SITL flight execution.

### Module Breakdown

- **Module 1: Depth/Point-Cloud to BEV Grid Encoder:** Convert raw front-facing depth camera maps or LiDAR point clouds into egocentric 2D Bird's-Eye-View (BEV) occupancy grids where each pixel cell encodes obstacle presence and elevation bounds.
- **Module 2: ConvLSTM Spatio-Temporal Predictor:** Pass consecutive BEV frames through ConvLSTM cells. The spatial convolutions retain localized obstacle geometry while recurrence updates temporal momentum, predicting where dynamic entities will move over the next 1–3 seconds.
- **Module 3: Recurrent Waypoint & Trajectory Controller:** A recurrent trajectory planner evaluates goal vector error $e_t = g - s_t$ alongside predicted future grid maps $\hat{O}_{t+1:t+p}$ to output smooth velocity commands.
- **Module 4: Safety Shield & PX4 SITL Flight Controller:** A low-latency safety controller (e.g., Dynamic Window Approach / Control Barrier Functions) overrides commands if predicted collision probability exceeds a safety threshold, transmitting finalized commands over MAVROS/MAVSDK to PX4 Autopilot.

## 4. Dataset Sources & Preprocessing Pipelines

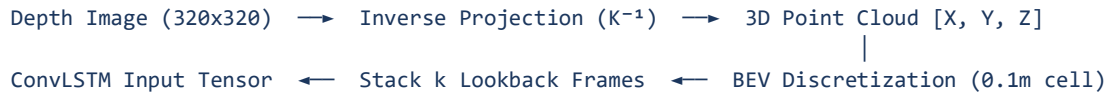
Students are provided three benchmark dataset options depending on computational resources:

| Dataset Name | Source / Platform | Key Features & Contents | Primary Project Use Case |
|---|---|---|---|
| AirSim Drone Flight Dataset | Kaggle (Public Repository) | 10,000 paired sequences of RGB images, 320x320 Depth maps (.npy), and telemetry velocity vectors. | Rapid offline ConvLSTM model prototyping and initial feature encoder pretraining. |
| CMU TartanAir SLAM Benchmark | CMU AirLab | Photorealistic sequences across extreme weather, dynamic objects, dense foliage, and ground-truth camera pose trajectories. | Training robust spatio-temporal models resilient to dynamic visual clutter and lighting changes. |
| MIT Blackbird UAV Dataset | MIT FAST Lab | Aggressive high-speed indoors flight data with millimeter-level optitrack motion capture ground truth. | Evaluating controller response accuracy and physical UAV kinodynamic constraint |

validation.

### Data Preprocessing Pipeline

To transform depth frame sequences into BEV ConvLSTM tensors, students must apply pinhole camera inverse projection:




## 5. Student Milestones & Project Deliverables

| Phase / Week | Milestone Task | Key Deliverables |
|---|---|---|
| Phase 1 (Weeks 1–2) | Dataset Setup & BEV Pipeline | PyTorch DataLoader converting depth images into (B, k, 1, H, W) BEV occupancy tensors. |
| Phase 2 (Weeks 3–4) | ConvLSTM Network Training | Trained model forecasting 5 future time steps. Achieve mIoU > 0.70 on test split. |
| Phase 3 (Weeks 5–6) | PX4 / AirSim SITL Integration | ROS 2 node subscribing to camera depth and outputting MAVROS velocity setpoints in SITL. |
| Phase 4 (Weeks 7–8) | Benchmark & Final Report | Comparative evaluation against static A* and reactive potential field baselines; Final Thesis Report. |

## 6. Quantitative Evaluation & Grading Criteria

Projects will be evaluated across two main categories using standardized metric benchmarks:

- **Perception Forecasting Accuracy:** Evaluated via Mean Intersection over Union (mIoU) across predicted future occupancy frames (Ô_{t+1:t+p}) vs ground truth grids, and Structural Similarity Index (SSIM).
- **Flight Navigation Performance:** Evaluated in SITL across 50 test runs via Mission Success Rate (%), Path Efficiency Ratio (L_actual / L_optimal), Minimum Safety Distance maintained (d_min), and Inference Latency (ms per control step).

> **Submission Checklist**
> 1. Clean PyTorch source code for ConvLSTM model and dataset preprocessors.
> 2. ROS 2 / SITL workspace package with launch scripts for Gazebo/AirSim environment.
> 3. Trained model weights (.pt / .onnx) along with training loss curves.
> 4. Formal M.Tech project report (IEEE format, maximum 10 pages) presenting qualitative flight trajectories and quantitative benchmark comparisons.